

```
"""
```

```
chat_voice - Notes vocales de la messagerie Jokoo
```

```
=====
```

Monté par server.py sous `/api` via `build\_chat\_voice\_router(...)` , sur le même patron que wallet / ambassadors / rides_v2.

Pourquoi un module séparé plutôt que du code dans server.py :
`server.py` est très fréquemment modifié (par plusieurs sessions en parallèle), et la zone `/chat/...` l'est particulièrement. Isoler les vocaux ici évite les conflits de merge à répétition et permet de faire évoluer la fonctionnalité sans toucher au cœur de l'API.

Décision d'architecture - l'audio n'est PAS inliné dans le message :
`GET /chat/{peer\_id}/messages` renvoie jusqu'à 1000 messages. Si chaque note vocale portait ~250 Ko de base64, une conversation active produirait une réponse de plusieurs dizaines de Mo - injouable sur la 3G sénégalaise, et proche de la limite BSON de 16 Mo par document. L'audio va donc dans la collection `chat\_media` et le message ne porte qu'un `media\_id` ; le client télécharge le son à la demande, au tap.

Endpoints :

POST /api/chat/{peer_id}/voice → envoie une note vocale

GET /api/chat/media/{media_id} → sert l'audio (accès restreint aux 2 pairs)

```
"""
```

```
from __future__ import annotations
```

```
import base64
```

```
import logging
```

```
import uuid
```

```
from datetime import datetime, timezone
```

```
from typing import Any, Callable, Literal, Optional
```

```
from fastapi import APIRouter, Depends, HTTPException, Response
```

```
from pydantic import BaseModel, Field
```

```
log = logging.getLogger("chat_voice")
```

```
# ~2 min d'AAC 24 kbps, marge incluse. Aligné avec la limite côté app.
```

```
MAX_VOICE_BYTES = 400_000
```

```
MAX_VOICE_SECONDS = 120
```

```

class VoiceMessageIn(BaseModel):
    """Note vocale envoyée par le client. `audio_b64` accepte un préfixe data:."""
    audio_b64: str = Field(min_length=16)
    duration_ms: int = Field(ge=500, le=MAX_VOICE_SECONDS * 1000)
    mime: Literal["audio/m4a", "audio/mp4", "audio/aac", "audio/mpeg"] = "audio/m

def _now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()

def _decode_voice_payload(audio_b64: str) -> bytes:
    """Décode et valide la taille d'une note vocale base64."""
    raw = (audio_b64 or "").strip()
    if raw.startswith("data:"):
        # data:audio/m4a;base64,XXXX
        _, _, raw = raw.partition(",")
    # Rejet précoce sur la taille encodée : évite de décoder 50 Mo pour rien.
    if len(raw) > MAX_VOICE_BYTES * 2:
        raise HTTPException(413, "Note vocale trop longue (2 min maximum).")
    try:
        audio = base64.b64decode(raw, validate=True)
    except Exception:
        raise HTTPException(400, "Audio invalide.")
    if not audio:
        raise HTTPException(400, "Audio vide.")
    if len(audio) > MAX_VOICE_BYTES:
        raise HTTPException(413, "Note vocale trop lourde (2 min maximum).")
    return audio

def build_chat_voice_router(
    db: Any,
    current_user: Callable,
    is_pair_blocked: Callable,
    conv_id_fn: Callable,
    send_push: Optional[Callable] = None,
) -> APIRouter:
    """
    Args:
        db : base Motor
        current_user : dépendance FastAPI -> dict utilisateur authentifié
        is_pair_blocked : async (a, b) -> bool - réutilise `_is_pair_blocked` de se
        conv_id_fn : (a, b) -> str - réutilise `_conv_id` de server.py
        send_push : async (recipients, data) -> None - optionnel, non bloquant
    """
    router = APIRouter(tags=["chat-voice"])

```

```

@router.post("/chat/{peer_id}/voice")
async def send_voice_message(
    peer_id: str,
    body: VoiceMessageIn,
    user=Depends(current_user),
):
    if await is_pair_blocked(user["id"], peer_id):
        raise HTTPException(
            403,
            "Impossible d'envoyer un message : vous ou l'autre partie l'avez
        )

    # Le destinataire peut être un user OU un provider (profils seedés).
    peer = await db.users.find_one({"id": peer_id}, {"_id": 0})
    if peer:
        peer_name = peer.get("name")
    else:
        prov = await db.providers.find_one({"id": peer_id}, {"_id": 0})
        if not prov:
            raise HTTPException(404, "Destinataire introuvable")
        peer_name = prov.get("name")

    audio = _decode_voice_payload(body.audio_b64)
    cid = conv_id_fn(user["id"], peer_id)
    media_id = str(uuid.uuid4())

    await db.chat_media.insert_one({
        "id": media_id,
        "conv_id": cid,
        # Participants stockés explicitement : `_conv_id` joint par "-" et le
        # ids sont des UUID (qui contiennent déjà des tirets), donc conv_id
        # n'est PAS parsable de façon fiable. Le contrôle d'accès en dépend.
        "participants": sorted([user["id"], peer_id]),
        "owner_id": user["id"],
        "kind": "voice",
        "mime": body.mime,
        "size_bytes": len(audio),
        "duration_ms": body.duration_ms,
        "data": audio, # bytes -> stocké en BinData, pas en base64
        "created_at": _now_iso(),
    })

    doc = {
        "id": str(uuid.uuid4()),
        "conv_id": cid,
        "from_id": user["id"],

```

```

        "from_name": user["name"],
        "to_id": peer_id,
        "to_name": peer_name,
        "text": "🎤 Note vocale",
        "kind": "voice",
        "media_id": media_id,
        "duration_ms": body.duration_ms,
        "mime": body.mime,
        "flagged": False,
        "flags": [],
        "read": False,
        "created_at": _now_iso(),
    }
    await db.messages.insert_one(doc)

    if peer:
        await db.notifications.insert_one({
            "id": str(uuid.uuid4()),
            "user_id": peer_id,
            "type": "message",
            "title": user["name"],
            "body": "🎤 Note vocale",
            "peer_id": user["id"],
            "read": False,
            "created_at": _now_iso(),
        })
    if send_push:
        try:
            await send_push(
                recipients=[peer_id],
                data={
                    "title": user["name"],
                    "message": "🎤 Note vocale",
                    "action_url": f"/chat/{user['id']}",
                },
            )
        except Exception as e:
            log.warning("Push failed (non-blocking): %s", e)

    return {k: v for k, v in doc.items() if k != "_id"}

@router.get("/chat/media/{media_id}")
async def get_chat_media(media_id: str, user=Depends(current_user)):
    """Sert un média de conversation.

    L'id du média ne suffit PAS : on revalide l'appartenance à la
    conversation, sinon toute personne connaissant un UUID pourrait écouter

```

```

    les vocaux d'autrui.
    """
    media = await db.chat_media.find_one({"id": media_id}, {"_id": 0})
    if not media:
        raise HTTPException(404, "Média introuvable")

    participants = media.get("participants") or []
    if not participants or user["id"] not in participants:
        # Doc hérité sans participants → on refuse plutôt que de deviner.
        raise HTTPException(403, "Accès refusé à ce média.")

    other = next((p for p in participants if p != user["id"]), None)
    if other and await is_pair_blocked(user["id"], other):
        raise HTTPException(403, "Accès refusé à ce média.")

    data = media.get("data") or b""
    if isinstance(data, str): # tolérance si un ancien doc stockait du base64
        try:
            data = base64.b64decode(data)
        except Exception:
            raise HTTPException(500, "Média illisible.")

    return Response(
        content=bytes(data),
        media_type=media.get("mime") or "audio/m4a",
        headers={"Cache-Control": "private, max-age=86400"},
    )

return router

async def ensure_indexes(db: Any) -> None:
    """Index recommandés. Idempotent – safe à appeler au démarrage."""
    try:
        await db.chat_media.create_index("id", unique=True, name="chat_media_id_u")
        await db.chat_media.create_index("participants")
        await db.chat_media.create_index([("conv_id", 1), ("created_at", -1)])
    except Exception as e:
        log.warning("chat_media ensure_indexes failed: %s", e)

```